VQ-VAE — Vector Quantized VAE

Replace the Gaussian latent with a learned dictionary. Discrete codes, no posterior collapse, and a clean split between learning a representation and learning a prior over it.

1 What problem does it attack?

Everything so far rests on one assumption — $q(z|x) = N(\mu_\phi, \sigma_\phi^2)$ — which brought three problems:

① POSTERIOR COLLAPSE

$\sigma \to 1$, $\mu \to 0$ is a **free** way to zero the KL term. The escape hatch is always open, and dimensions take it whenever reconstruction does not object.

② BLUR

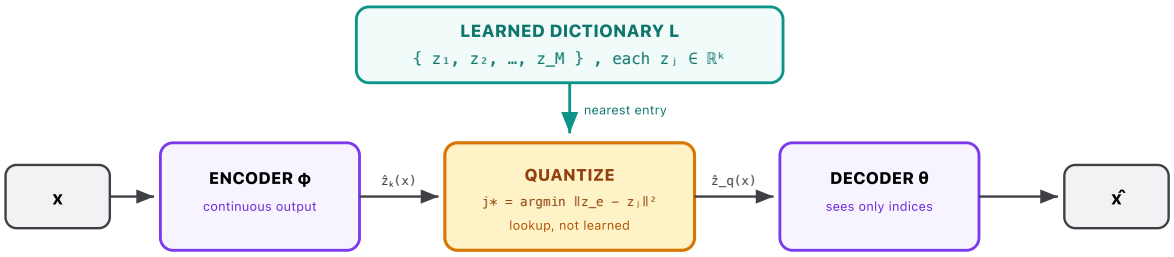
Gaussian likelihood → $-\|x-\hat{x}\|^2$ → MSE. The optimum of MSE is the **mean**, and the mean of five different "7"s is a smudge.

③ MUSHY CODE

A continuous cloud is an awkward fit for genuinely **categorical** structure — words, phonemes, class labels, discrete attributes.

VQ-VAE's answer: make the latent **discrete** — not by rounding, but by introducing an explicit **learned dictionary** and snapping the encoder's output onto it.

2 The architecture



The **only new object** is the codebook L. Everything else is an ordinary autoencoder.

3 The quantisation step, precisely

For **each** of the p vectors in the encoder output (your notes: $\hat{z}_e(x_i) \in \mathbb{R}^{k \times t}$ — p vectors of dimension k):

```
j*  =  argmin_{j=1..M}  ‖ z_e(x_i) − z_j ‖²        ← find the nearest dictionary entry
z_q(x_i)  =  z_{j*}                                ← replace with it
```

⚠ **The continuous vector is discarded.** What passes to the decoder is only a discrete **index** — *which* of the M entries, one per position. All the fine-grained continuous information is thrown away.

4 💥 The new problem: argmin kills the gradient

`argmin` is a discrete selection — a staircase, not a slope. Ask *"if I nudge z_e, how does z_q change?"* and the answer is **it doesn't**, until you cross a boundary and then it jumps entirely.

```
∂z_q / ∂z_e  =  0     almost everywhere
∴  the gradient dies at the quantiser.  The DECODER learns fine.
    The ENCODER learns NOTHING — no gradient can cross argmin.
```

This is **structurally identical to Phase D**. There, an opaque `sample()` severed the path. Here, an opaque `argmin` severs it. Same disease — but VQ-VAE prescribes a **completely different medicine**.

5 The fix: straight-through + codebook loss

We cannot differentiate the quantiser honestly, so we **lie about it** — precisely and controllably.

1 The straight-through estimator

```
FORWARD   :  z_q = z_{j*}                          (real quantisation, no approximation)
BACKWARD  :  ∂L/∂z_e  :=  ∂L/∂z_q                  (fake it — "straight through")
```

We tell the encoder: *"behave as though your output had passed through unchanged."* Mathematically wrong, and it works well — because the encoder only needs a sensible **direction**, and copying the decoder's gradient gives it one.

2 Move the dictionary instead (this is in your notes)

```
‖ z_e(x) − z_q(x) ‖²        ← pulls the CHOSEN dictionary entry toward the encoder output
```

Without this the encoder drifts and the dictionary never follows.

3 The commitment term (not in your notes)

```
β · ‖ z_e(x) − z_q(x) ‖²      ← pulls z_e toward z_q:  "commit to your choice"
```

Without it, z_e can grow arbitrarily large while the dictionary chases it. **This β is not β-VAE's β** — same symbol, unrelated job.

6 The loss and dictionary learning

```
J(θ,φ) = ‖ x − x̂(ẑ_q) ‖²  +  ‖ z_e(x_i) − z_q(x) ‖²
          ↑ reconstruction        ↑ codebook loss
z_j^{t+1} ← z_j^{t} − α ∇_{z_j}( J(θ,φ) )
```

And a crucial point from your notes (p70): **"in addition to the Encoder & Decoder parameters, the latent dictionary is also learned."**

Only the **chosen** entry receives a gradient. The other M−1 entries get **zero** and sit still at this step. Over the dataset, different inputs select different entries, and each migrates toward the region of encoder-space that selects it.

→ The dictionary is not a fixed grid. It is **M points learning to tile the encoder's output space** — a Voronoi diagram being fitted to the data.

7 Inference (p70)

```
VAE    :  embedding of x  =  μφ(x)                              (a continuous point)
VQ-VAE :  embedding of x  =  argmin_j ‖ ẑ_e(x) − z_j ‖²      (a discrete INDEX)
```

Your "feature vector" is now a list of **p integers**, each between 1 and M.

8 Sampling: the decoupling insight (p71)

Problem: to generate you must feed the decoder a z_q — but z_q must be *one of the dictionary entries*. Where does one come from? Your notes:

> *"To learn to sample from the dist. of ẑ_q(·), a generative model is fit to samples of z_q. Eg: GMM on the samples from the o/p of Encoder corresponding to all #p datapoints."*

```
STAGE 1 :  train encoder + decoder + codebook     → learn a good DISCRETE representation
STAGE 2 :  fit a prior over the code indices      → learn which codes co-occur
           (GMM per your notes; PixelCNN / autoregressive in the paper)

GENERATE:  sample code indices from Stage 2  →  decode  →  new x
```

In a normal VAE, modelling the data and modelling the prior are **entangled** inside one loss. VQ-VAE **separates** them — and since the codes are discrete, Stage 2 can be *any* model, including powerful autoregressive ones.

This is the design pattern behind much of modern generative modelling: **tokenise the data, then model the tokens.**

9 What it fixes, and what it breaks

	continuous VAE	VQ-VAE
posterior collapse (σ→1)	**possible — free cheat exists**	**structurally impossible**
differentiability	reparameterization trick	straight-through estimator
latent	continuous cloud	discrete index
prior modelling	entangled in the loss	**decoupled — any model**
its own failure mode	—	**codebook collapse**

Why continuous collapse is impossible here: the free cheat was μ=0, σ=1. VQ-VAE has **no μ and no σ**. To make z_q uninformative every code would have to be identical — which reconstruction fights directly. There is no silent, zero-cost way to switch the code off.

But it has a cousin: codebook collapse. Because only the *nearest* code gets a gradient, the encoder can drift into using a handful of entries while the rest sit dead — wasted capacity. Standard fixes: the commitment loss, EMA codebook updates, and restarting dead entries.
Honest note: your notes cover the codebook loss but not the straight-through estimator, the commitment term, or codebook collapse. All three are needed to actually train one.

10 Why the discrete code is so attractive

```
cost = p × log₂(M) bits       p=32, M=512  →  32 × 9  =  288 bits
vs 28×28 greyscale = 6,272 bits      →  ~22× compression
```

Every bit is a **learned, semantically meaningful choice** rather than a raw pixel.

One-line summary. VQ-VAE replaces the Gaussian latent with a learned dictionary. `argmin` kills the gradient, so it uses a straight-through estimator for the encoder and a codebook loss to learn the dictionary — and in return gets a discrete, compact code with no posterior collapse, plus a clean split between learning the representation and learning the prior over it.